Reachy_Local_08.4.2 Project Completion Plan

Date: 2025-11-03 **Version**: 1.0

Table of Contents

1. Executive Summary
2. Current State Assessment
3. System Architecture Overview
4. High-Level Completion Plan
5. Detailed Implementation Plan
6. Placeholder Definitions
7. Risk Mitigation
8. Success Criteria

Executive Summary

Reachy_Local_08.4.2 is a privacy-first robotic emotion recognition system combining synthetic video generation, human-in-the-loop labeling, continuous model improvement, and real-time edge inference. The project integrates three primary computing nodes (Ubuntu 1, Ubuntu 2, and Jetson Xavier NX) to deliver an end-to-end emotion classification pipeline for the Reachy Mini companion robot.

Primary Goal: Deploy a production-ready emotion recognition system that achieves 95% accuracy while maintaining sub-100ms response times and strict privacy guarantees.

Project Timeline: Target production release by May 10, 2026, with beta release February 15, 2026.

Current State Assessment

Completed Components (90% Backend Infrastructure)

1. **Ubuntu 1 (Model Host) - Backend Services**
   - PostgreSQL 16 database schema with video metadata tables
   - FastAPI Media Mover service (`/api/media/*` endpoints)
   - Promotion logic with staging to `/videos/dataset_all/`
   - Manifest rebuild functionality
   - Dry-run support for safe operations
   - Observability instrumentation (Prometheus metrics)
   - File system integration with atomic operations

2. **Network Configuration**
   - Static IP assignments (Ubuntu 1: 10.0.4.130, Ubuntu 2: 10.0.4.140, Jetson: 10.0.4.150)
   - Nginx reverse proxy configuration
   - API gateway routing established
   - CORS and security headers configured
3. **Project Structure**
   - Comprehensive requirements (v0.08.4.3) with acceptance criteria
   - Nine-agent agentic AI system design documented
   - Memory-bank system for persistent context
   - Decision records and runbook templates

In-Progress Components (40% Implementation)

1. **Web Application (Streamlit)**
   - Basic UI structure with landing page
   - Video upload/generation interface design
   - Emotion classification form (6-class taxonomy)
   - **Missing**: API integration, real-time updates, video streaming
2. **API Integration Layer**
   - Gateway API structure defined
   - Pydantic v2 schemas created
   - **Missing**: Live connections between UI and backend services

Not Started Components (0% Implementation)

1. **Machine Learning Pipeline**
   - TAO Toolkit environment setup
   - EmotionNet model fine-tuning workflow
   - Training data preparation scripts
   - Model evaluation and validation
2. **Jetson Edge Deployment**
   - DeepStream pipeline configuration
   - TensorRT engine optimization
   - Real-time inference implementation
   - WebSocket communication with Ubuntu 2
3. **Orchestration & Automation**
   - n8n workflow configurations for 9 agents
   - Agent-to-agent communication protocols
   - Automated promotion and training triggers
4. **Video Generation Integration**
   - Luma/Runway API integration
   - Synthetic video pipeline
   - Prompt templates and management
5. **Monitoring & Observability**
   - Grafana dashboards

- Alert rules and thresholds
- Log aggregation pipeline
- Performance metrics collection

System Architecture Overview

Current Network Topology

```
+----------------------+     +----------------------+     +----------------------+
| Ubuntu 1 (130)       | --> | Ubuntu 2 (140)       | <-- | Jetson NX (150)      |
| Model Host           |     | App Gateway          |     | Edge Device          |
| - PostgreSQL DB      |     | - Nginx Proxy        |     | - DeepStream         |
| - Media Mover API    |     | - FastAPI Gateway    |     | - TensorRT           |
| - TAO Training       |     | - Streamlit UI       |     | - Camera Input       |
| - LM Studio (LLM)    |     | - n8n Orchestrator   |     | - WebSocket Client   |
| - Video Storage      |     | - WebSocket Server   |     | - Emotion Engine     |
| - MLflow Tracking    |     |                      |     |                      |
+----------------------+     +----------------------+     +----------------------+
```

Data Flow Architecture

```
1. Video Generation -> Ingest -> Label -> Promote -> Dataset_all
2. Dataset_all -> Random Sample -> Train/Test Splits -> TAO Training
3. Model -> TensorRT Export -> DeepStream Deploy -> Jetson Inference
4. Live Camera -> Emotion Detection -> LLM Adaptation -> User Response
```

Storage Layout (`/media/project_data/reachy_emotion/videos/`)

```
videos/
|- temp/           # Unlabeled incoming videos (7-14 day TTL)
|- dataset_all/    # Canonical labeled corpus (permanent)
|- train/          # Per-run training subset (70% random sample)
|- test/           # Per-run test subset (30% random sample, unlabeled)
|- thumbs/         # Video thumbnails (JPEG)
\- manifests/      # JSONL manifests with run_id tracking
```

High-Level Completion Plan

Phase 1: Foundation Completion (Weeks 1-2)

Objective: Complete all backend infrastructure and establish end-to-end connectivity

Priority Actions:

1. **Finalize Web UI Integration**
   - Connect Streamlit to Media Mover API
   - Implement video upload/streaming
   - Add real-time WebSocket updates
   - Complete emotion labeling workflow
2. **Complete Database Integration**
   - Finalize training_run and selection tables
   - Implement run_id based sampling
   - Add audit logging tables
   - Create backup/restore procedures

Phase 2: ML Pipeline (Weeks 3-5)

Objective: Establish complete training and evaluation pipeline

Priority Actions:

1. **TAO Environment Setup**
   - Configure TAO 4.x container for training
   - Setup TAO 5.3 for TensorRT export
   - Create emotion_train_2cls.yaml specification
   - Implement MLflow experiment tracking
2. **Training Orchestration**
   - Build dataset preparation scripts
   - Implement balanced sampling logic
   - Create training automation workflows
   - Establish Gate A/B/C validation

Phase 3: Edge Deployment (Weeks 6-7)

Objective: Deploy real-time inference on Jetson

Priority Actions:

1. **DeepStream Configuration**
   - Setup gst-nvinfer pipeline
   - Configure TensorRT optimization
   - Implement FPS/latency monitoring
   - Create rollback procedures
2. **Jetson Integration**
   - Establish WebSocket communication
   - Implement emotion event streaming
   - Add camera preprocessing pipeline
   - Configure edge security

Phase 4: Orchestration & Automation (Weeks 8-9)

Objective: Automate all workflows via n8n agents

Priority Actions:

1. **Agent Implementation**
   - Configure all 9 n8n agent workflows
   - Setup inter-agent messaging
   - Implement retry/circuit breaker logic
   - Add dead letter queue handling
2. **Video Generation Pipeline**
   - Integrate Luma/Runway APIs
   - Build prompt template system
   - Implement generation balancer
   - Add synthetic metadata tracking

Phase 5: Production Readiness (Weeks 10-12)

Objective: Harden system for production deployment

Priority Actions:

1. **Monitoring & Observability**
   - Deploy Grafana dashboards
   - Configure Prometheus alerts
   - Setup log aggregation
   - Implement SLA tracking
2. **Security & Compliance**
   - Implement JWT authentication
   - Configure mTLS for services
   - Add rate limiting
   - Complete privacy audits

Detailed Implementation Plan

Week 1-2: Foundation Completion

1.1 Web Application Integration

- ☐ **Task 1.1.1**: Connect Streamlit to Media Mover API
  - Sub-task: Implement authentication headers
  - Sub-task: Add request retry logic
  - Sub-task: Handle API error responses
- ☐ **Task 1.1.2**: Implement video streaming from Ubuntu 1
  - Sub-task: Configure Nginx for video serving
  - Sub-task: Add range request support

  - Sub-task: Implement thumbnail display
- ☐ **Task 1.1.3**: Add WebSocket support for real-time updates
  - Sub-task: Create WebSocket client in Streamlit
  - Sub-task: Handle connection lifecycle
  - Sub-task: Implement message queuing
- ☐ **Task 1.1.4**: Complete emotion labeling workflow
  - Sub-task: Add label validation
  - Sub-task: Implement promotion confirmation
  - Sub-task: Add batch operations support

1.2 Database Finalization

- ☐ **Task 1.2.1**: Complete schema updates

```
-- Add training_run and training_selection tables
-- Add promotion_log table for audit trail
-- Add user_session table for tracking
```

- ☐ **Task 1.2.2**: Implement database procedures
  - Sub-task: Create promotion stored procedures
  - Sub-task: Add trigger for updated_at timestamps
  - Sub-task: Implement cascade deletes
- ☐ **Task 1.2.3**: Setup backup automation
  - Sub-task: Configure pg_dump cron job
  - Sub-task: Implement NAS sync for backups
  - Sub-task: Create restore verification script

Week 3-5: ML Pipeline

2.1 TAO Training Environment

- ☐ **Task 2.1.1**: Setup TAO 4.x container

```
docker pull nvcr.io/nvidia/tao/tao-toolkit:4.0.0-tf2.11.0
```

  - Sub-task: Configure GPU passthrough
  - Sub-task: Mount dataset volumes
  - Sub-task: Setup environment variables
- ☐ **Task 2.1.2**: Create training specifications

```yaml
# emotion_train_2cls.yaml
model_config:
  arch: "resnet18"
  num_classes: 2  # happy, sad
  input_shape: [224, 224, 3]
```

- ☐ **Task 2.1.3**: Implement data augmentation pipeline

- Sub-task: Add random crops/flips
- Sub-task: Configure color jitter
- Sub-task: Implement mixup strategy

2.2 Training Automation

☐ **Task 2.2.1**: Build dataset preparation scripts

```
# prepare_dataset.py
def sample_balanced_dataset(run_id, train_ratio=0.7):
    # Copy from dataset_all to train/test
    # Ensure label balance
    # Generate manifests
```

☐ **Task 2.2.2**: Create MLflow integration

- Sub-task: Log hyperparameters
- Sub-task: Track metrics per epoch
- Sub-task: Store model artifacts
- Sub-task: Link dataset_hash

☐ **Task 2.2.3**: Implement validation gates

- Sub-task: Gate A - Offline validation (F1 >= 0.84)
- Sub-task: Gate B - Shadow mode (latency <= 250ms)
- Sub-task: Gate C - User rollout (complaints < 1%)

Week 6-7: Edge Deployment

3.1 DeepStream Setup

☐ **Task 3.1.1**: Configure DeepStream pipeline

```
<!-- deepstream_config.txt -->
[source0]
type=1
camera-width=1920
camera-height=1080
camera-fps-n=30
```

☐ **Task 3.1.2**: Optimize TensorRT engine

- Sub-task: Export model with FP16 precision
- Sub-task: Generate INT8 calibration data
- Sub-task: Profile inference performance

☐ **Task 3.1.3**: Implement preprocessing pipeline

- Sub-task: Add face detection stage
- Sub-task: Configure ROI extraction
- Sub-task: Implement sliding window

3.2 Jetson Integration

- ☐ **Task 3.2.1**: Setup WebSocket client

```python
# jetson_client.py
async def send_emotion_event(emotion, confidence):
    payload = {
        "device_id": DEVICE_ID,
        "emotion": emotion,
        "confidence": confidence,
        "inference_ms": latency
    }
```

- ☐ **Task 3.2.2**: Configure system services
  - Sub-task: Create systemd service for DeepStream
  - Sub-task: Add auto-restart on failure
  - Sub-task: Setup log rotation
- ☐ **Task 3.2.3**: Implement monitoring
  - Sub-task: Track GPU utilization
  - Sub-task: Monitor thermal throttling
  - Sub-task: Log FPS and latency metrics

Week 8-9: Orchestration & Automation

4.1 n8n Agent Implementation

- ☐ **Task 4.1.1**: Ingest Agent

```json
{
  "workflow": "ingest_agent",
  "triggers": ["video_upload", "video_generated"],
  "actions": ["compute_hash", "extract_metadata", "generate_thumb"]
}
```

- ☐ **Task 4.1.2**: Labeling Agent
  - Sub-task: Handle label events from UI
  - Sub-task: Validate label constraints
  - Sub-task: Update class balance counters
- ☐ **Task 4.1.3**: Promotion Agent
  - Sub-task: Implement atomic file moves
  - Sub-task: Update database transactionally
  - Sub-task: Trigger manifest rebuild
- ☐ **Task 4.1.4**: Training Orchestrator
  - Sub-task: Monitor dataset readiness
  - Sub-task: Launch TAO training jobs

- Sub-task: Handle training completion

☐ **Task 4.1.5**: Remaining Agents

- Reconciler Agent (consistency checks)
- Evaluation Agent (test set validation)
- Deployment Agent (model rollout)
- Privacy Agent (TTL enforcement)
- Observability Agent (metrics aggregation)

4.2 Video Generation Integration

☐ **Task 4.2.1**: Luma API Integration

```python
# luma_client.py
async def generate_video(prompt):
    response = await luma_api.create_generation({
        "prompt": prompt,
        "aspect_ratio": "16:9",
        "duration": 5
    })
```

☐ **Task 4.2.2**: Prompt Template System

- Sub-task: Create emotion-specific templates
- Sub-task: Add scene variation logic
- Sub-task: Implement diversity controls

☐ **Task 4.2.3**: Generation Balancer

- Sub-task: Track class distribution
- Sub-task: Bias generation towards minority class
- Sub-task: Implement quota system

Week 10-12: Production Hardening

5.1 Monitoring Infrastructure

☐ **Task 5.1.1**: Deploy Grafana

```yaml
# docker-compose.yml
grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  volumes:
    - ./dashboards:/etc/grafana/dashboards
```

☐ **Task 5.1.2**: Create dashboards

- System Health Dashboard
- ML Pipeline Dashboard

- User Activity Dashboard
- Error Rate Dashboard

☐ **Task 5.1.3**: Configure alerts

- High error rate (> 1%)
- Low accuracy (< 80%)
- High latency (> 300ms)
- Storage capacity (> 80%)

5.2 Security Implementation

☐ **Task 5.2.1**: JWT Authentication

```python
# auth_middleware.py
def verify_jwt_token(token: str):
    payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    return payload
```

☐ **Task 5.2.2**: Service hardening

- Sub-task: Enable mTLS between services
- Sub-task: Implement rate limiting
- Sub-task: Add request validation
- Sub-task: Configure firewall rules

☐ **Task 5.2.3**: Privacy compliance

- Sub-task: Implement data purge API
- Sub-task: Add consent tracking
- Sub-task: Create DSAR process
- Sub-task: Document retention policies

Placeholder Definitions

Configuration Placeholders

Placeholder	Purpose	Default Value	Location
`{LUMA_API_KEY}`	Luma video generation API key	Environment variable	`.env`
`{JWT_SECRET}`	JWT signing secret	Random 256-bit key	Vault
`{DB_PASSWORD}`	PostgreSQL password	Environment variable	`.env`
`{MLFLOW_TRACKING_URI}`	MLflow server URL	`http://localhost:5000`	Config

Placeholder	Purpose	Default Value	Location
`{DEVICE_ID}`	Jetson device identifier	`reachy-mini-01`	Config
`{N8N_WEBHOOK_URL}`	n8n webhook endpoint	`http://localhost:5678/webhook`	Config
`{GRAFANA_API_KEY}`	Grafana API key	Generated on setup	Vault
`{TAO_API_KEY}`	NVIDIA TAO API key	NGC credentials	`.ngc`

Code Placeholders

Placeholder	Purpose	Implementation Notes
`# TODO: Video generation client`	Luma/Runway API client	Implement async client with retry logic
`# TODO: WebSocket handler`	Real-time communication	Use python-socketio with reconnection
`# TODO: Training script`	TAO training launcher	Shell script wrapping TAO CLI
`# TODO: Export script`	TensorRT conversion	TAO 5.3 export with optimization
`# TODO: Backup script`	Database/media backup	Cron job with verification
`# TODO: Monitoring collector`	Metrics collection	Prometheus client instrumentation

Infrastructure Placeholders

Placeholder	Purpose	Deployment Target
`{NAS_MOUNT_POINT}`	Network storage mount	`/mnt/synology`
`{VIDEO_STORAGE_ROOT}`	Video storage path	`/media/project_data/reachy_emotion/videos`
`{MODEL_STORAGE_PATH}`	Model artifacts path	`/models/emotionnet`
`{LOG_STORAGE_PATH}`	Log aggregation path	`/var/log/reachy`
`{DOCKER_REGISTRY}`	Container registry	`ghcr.io/reachy-emotion`

Risk Mitigation

Technical Risks

Risk	Impact	Mitigation Strategy
Model accuracy below target	High	Implement active learning loop, increase dataset size
Inference latency exceeds 100ms	High	Use INT8 quantization, optimize pipeline
Storage capacity exhaustion	Medium	Implement automated pruning, NAS overflow
Network connectivity issues	Medium	Add local caching, implement retry logic
GPU thermal throttling	Medium	Add cooling, implement dynamic batching

Operational Risks

Risk	Impact	Mitigation Strategy
Data loss	High	Daily backups, NAS replication, ZFS snapshots
Service downtime	High	Implement health checks, auto-restart, rollback
Security breach	High	mTLS, JWT rotation, network segmentation
Drift between environments	Medium	Version pinning, reproducible builds
Agent orchestration failure	Medium	Dead letter queues, manual override capability

---

Success Criteria

Phase 1 Success Metrics

- ☒ All backend APIs return 200 status

- ☒ Web UI can upload and label videos
- ☒ Database contains > 100 labeled videos
- ☒ Promotion workflow moves files correctly
- ☒ Manifests rebuild within 2 minutes

Phase 2 Success Metrics

- ☒ TAO training completes successfully
- ☒ Model achieves > 84% F1 score
- ☒ MLflow tracks all experiments
- ☒ TensorRT engine exports correctly
- ☒ Validation gates pass automatically

Phase 3 Success Metrics

- ☒ DeepStream runs at 30 FPS
- ☒ Inference latency < 100ms (p50)
- ☒ WebSocket events stream reliably
- ☒ Emotion detection accuracy > 85%
- ☒ System runs 24 hours without crash

Phase 4 Success Metrics

- ☒ All 9 agents operational in n8n
- ☒ Video generation produces 50 clips/day
- ☒ Class balance maintained at 50/50 +/-5%
- ☒ Automated training triggers work
- ☒ End-to-end pipeline < 10 min/video

Phase 5 Success Metrics

- ☒ 99.9% uptime over 7 days
- ☒ All dashboards populated with data
- ☒ Alerts fire correctly on issues
- ☒ Security audit passes
- ☒ Documentation complete

Next Steps

Immediate Actions (This Week)

1. Complete Web UI to API integration
2. Test end-to-end promotion workflow
3. Setup TAO 4.x development environment
4. Create first n8n agent workflow (Ingest)
5. Deploy basic Prometheus/Grafana stack

Critical Path Items

1. TAO training pipeline (blocks all ML work)
2. WebSocket implementation (blocks real-time features)
3. n8n orchestration (blocks automation)
4. Jetson deployment (blocks edge testing)
5. Monitoring setup (blocks production readiness)

Dependencies

- Luma API access (for video generation)
- NVIDIA NGC account (for TAO containers)
- Synology NAS configuration (for backups)
- SSL certificates (for mTLS)
- Production hardware (Jetson Xavier NX)

Conclusion

The Reachy_Local_08.4.2 project has a solid foundation with 90% of backend infrastructure complete. The primary focus should shift to:

1. **Immediate**: Completing Web UI integration to enable data collection
2. **Short-term**: Establishing the ML training pipeline for model development
3. **Medium-term**: Deploying edge inference and orchestration automation
4. **Long-term**: Hardening for production with monitoring and security

With focused execution on the detailed plan above, the project can achieve beta release by February 2026 and production deployment by May 2026.

Document Version: 1.0
Last Updated: 2025-11-03
Author: Cascade AI Assistant
Review Status: Ready for review